

Task 1

7 14

Task 2

```
numbers = [10, 20, 30, 40, 50]
total = 0
for i in range(len(numbers)):
    total += numbers[i]
print(total)
```

It calculates the last number (50).
Intended: calculate the sum.

Task 3

- a) 100 times
- b) 1,000,000 times
- c) 10,000,000 times

Task 4

Algorithm A: $O(n)$
Algorithm B: $O(n^2)$
B grows much faster.
For $n = 1,000,000$: $A \approx 1$ million operations; $B \approx 1$ trillion.

Task 5

$2^n > n^2 > n \cdot \log_2 n > n > \log_2 n > 1$

Most problematic: 2^n and n^2 .

Task 6

Unsorted $\rightarrow$ Linear search: $O(n)$
Sorted $\rightarrow$ Binary search: $O(\log n)$

Binary search is faster because it eliminates half the search space each step.

Task 7

Keep one variable max. Examine each number once.

Time: $O(n)$
Memory: $O(1)$
No need to store all numbers.

Yes, process in chunks.

Task 8

Use a dictionary/hash map.

A → 4

B → 3

C → 2

D → 1

```
def count_frequencies(sequence):
    # Step 1: Initialize hash map
    freq_map = {}

    # Step 2: Count occurrences
    for item in sequence:
        freq_map[item] = freq_map.get(item, 0) + 1

    # Step 3: Sort by frequency in descending order
    sorted_counts = sorted(freq_map.items(), key=lambda x: x[1], reverse=True)

    # Step 4: Format output
    return ", ".join(f"{k}→{v}" for k, v in sorted_counts)

# Input
data = ["A", "B", "A", "C", "B", "A", "D", "C", "A", "B"]

# Execution
result = count_frequencies(data)
print(result)
```

Task 9

$A + B = [[6,8],$
 $[10,12]]$

$A \times B = [[19,22],$
 $[43,50]]$

Multiplications: 8

Task 10

Possible reasons:

Memory bandwidth is slow.

Cache misses.

Large amount of data / I/O bottleneck.

Task 11

Command → OS loads program → CPU executes instructions
→ program processes data → output is produced.

Task 12

A → CPU bottleneck
B → Memory bottleneck
C → Disk/I/O bottleneck
D → Waiting for input

Yes, they can run simultaneously. CPU time depends on scheduling and available CPU cores. With one core, programs share the CPU through time-slicing.

Task 13

No. Not necessarily. The program may have I/O, memory, or other bottlenecks. A 2× faster CPU does not guarantee 2× faster execution.

Task 14

Possible ways:

Better algorithm
Reduce unnecessary calculations
Use efficient data structures
Parallelize work
Use caching
Reduce I/O
Optimize memory usage
Use faster hardware
Compile/optimize the code

Task 15

Original: 59 seconds

Optimized:

A = 0.2s
B = 0.2s
C = 50s
D = 0.3s
E = 0.2s
Total = 50.9s

Speedup:

$$59 / 50.9 \approx 1.16\times$$

Focus optimization on C, because it takes 50 seconds.

Task 16

Use:

One pass to calculate min, max, average, ≥ 90 , < 40

Store marks or use a frequency array of 101 values to calculate the median.

Time: $O(n)$

Memory: $O(n)$ if storing marks

$O(101)$ using frequency counts

The data is examined about once (plus a small 101-element scan for the median).

Likely bottleneck: reading/processing 10 million marks.

Make it faster using parallel processing/chunking.